

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Multimodal Generative AI Applications

(Text-to-Image Generation and Speech-to-Text Word Counting)

Course Code:	CSA6502	Course Title:	Generative AI and Large Scale Model
CO Assessed:	CO4 – Precision(BL3, BL4, BL5)	Assessment:	Assessment Tool 3 - Multimodal Mini-Demo (Text-to-Image + Speech-to-Text/Text-to-Speech)
Weightage:	10% of CIA	Total Marks:	2*50=100
CO4 Statement: Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.(BL3, BL4,BL5)			
Student Name:	C. Praveen kumar	Reg. No.:	192472034
Section / Batch:	2024	Date:	26-08-2026

Code of Conduct

I, C. Praveen kumar / 192472034 (Name / Reg No), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: C. Praveen kumar

Name: C. Praveen kumar Reg.No: 192472034

1. Question 1 — Text-to-Image Generator (“A Cat Playing with a Robot”)

1.1 Aim / Objective

To develop a Python program that accepts a short natural-language description from the user and generates a corresponding image using a pre-trained text-to-image generative AI model, without training any model from scratch.

- To accept a free-text image description as input, e.g. “a cat playing with a robot”.
- To use a pre-trained diffusion-based text-to-image model through a hosted inference API.
- To display the generated image to the user through a simple web interface.
- To handle invalid or empty input gracefully, without crashing the application.

1.2 Problem Statement

Manually creating a custom image for a given idea requires artistic skill, design software, and considerable time. Many practical situations — quick concept art, presentation visuals, creative brainstorming, or illustrating an idea in a document — need a fast way to turn a plain-English description into a usable image. The problem addressed here is to build a lightweight Python application that takes a short text prompt and automatically produces a relevant, reasonably accurate image, using an existing pre-trained generative model rather than building or training a model in-house.

1.3 Requirements

Software Requirements:

- Python 3.9 or later
- Streamlit (for the web interface)
- Requests library (for calling the inference API)
- Pillow / PIL (for handling and displaying the returned image)
- A Hugging Face account and a free API access token

Hardware Requirements:

- A computer or laptop with a stable internet connection
- Minimum 4 GB RAM (no local GPU is required, since inference runs on the hosted API)

1.4 Selected AI Model / API and Justification

Selected model: Stable Diffusion (accessed through the Hugging Face Inference API).

Justification:

- Stable Diffusion is a widely used, well-documented diffusion-based text-to-image model that produces high-quality images from natural-language prompts.
- The Hugging Face Inference API removes the need for a local GPU or a large model download, which makes the application lightweight and easy to run on a standard laptop.
- A free-tier API token is sufficient for development, testing, and academic demonstration.
- The API returns a ready-to-use image (bytes), which keeps the client-side Python code short and easy to maintain.

1.5 Architecture / Workflow

User enters a text prompt in the Streamlit interface → Input is validated (checked for empty/invalid text) → A POST request is sent to the Hugging Face Inference API with the prompt → The pre-trained Stable Diffusion model generates the image on the server side → The API returns the image as binary (JPEG/PNG) data → The Python program reads the bytes into a PIL Image object → The generated image is displayed back to the user in the browser.

1.6 Algorithm / Working Principle

- Step 1: Start the Streamlit application and display a text input box to the user.
- Step 2: Read the text entered by the user as the image prompt.
- Step 3: Validate the prompt — if it is empty or contains only whitespace, show a warning and stop, without calling the API.
- Step 4: If the prompt is valid, send it to the Hugging Face Inference API endpoint for the chosen Stable Diffusion model, along with the authorization header.
- Step 5: Wait for the API response while showing a loading indicator to the user.
- Step 6: If the response status is successful (HTTP 200), convert the returned bytes into an image object and display it.
- Step 7: If the response indicates an error (invalid token, model loading, rate limit, etc.), display a clear error message instead of the image.

1.7 Python Implementation

```
import streamlit as st
import requests
from PIL import Image
import io

st.set_page_config(page_title="AI Text-to-Image Generator")
st.title("AI Text-to-Image Generator")
st.write("Enter a short description and generate an image using a pre-trained "
```

```

        "text-to-image model.")

API_URL = "https://api-inference.huggingface.co/models/stabilityai/stable-diffusion-2-1"
HF_TOKEN = "YOUR_HUGGINGFACE_API_TOKEN" # replace with your own token
headers = {"Authorization": f"Bearer {HF_TOKEN}"}

def generate_image(prompt: str):
    """Send the prompt to the Hugging Face Inference API and return a PIL Image,
    or None if the request failed."""
    response = requests.post(API_URL, headers=headers, json={"inputs": prompt}, timeout=60)
    if response.status_code == 200:
        return Image.open(io.BytesIO(response.content))
    return None

prompt = st.text_area(
    "Enter image description:",
    placeholder="e.g. a cat playing with a robot"
)

if st.button("Generate Image"):
    cleaned_prompt = prompt.strip() if prompt else ""

    if cleaned_prompt == "":
        st.warning("Please enter a valid description before generating an image.")
    else:
        with st.spinner("Generating image..."):
            image = generate_image(cleaned_prompt)

        if image is not None:
            st.success("Image generated successfully!")
            st.image(image, caption=cleaned_prompt, use_column_width=True)
        else:
            st.error("Image generation failed. Please check your API token, "
                    "internet connection, or try again in a few seconds "
                    "(the model may still be loading on the server).")

```

1.8 Input → Processing → Generated Image

Example walkthrough using the prompt “a cat playing with a robot”:

- Input: the user types “a cat playing with a robot” into the text box and clicks “Generate Image”.
- Processing: the program strips extra whitespace, confirms the text is non-empty, and sends it as JSON to the Hugging Face Inference API.
- Output: the API returns image bytes representing a cat interacting with a robot; the bytes are decoded into a PIL Image and rendered on the page with the original prompt as the caption.

1.9 Test Cases

Test Case	Input Prompt	Expected Behaviour	Result
TC1	“a cat playing with a robot”	Generates an image showing a cat and a robot interacting	Pass
TC2	“a futuristic city skyline at sunset”	Generates a cityscape image with warm sunset lighting	Pass
TC3	“an astronaut riding a horse on Mars”	Generates a surreal image combining all three elements	Pass
TC4	“xyzxyz123!!!” (nonsensical text)	Model still returns some image, though not semantically meaningful	Pass (as expected)
TC5	“” (empty input)	Warning shown; no API call made	Pass

1.10 Sample Outputs / Screenshots



1.11 Invalid / Empty Input Handling

The application checks the prompt text before making any API call:

- Empty input: if the text box is empty or contains only spaces, a warning (“Please enter a valid description...”) is shown and no request is sent, avoiding a wasted API call.
- API/network failure: if the request times out, the token is invalid, or the model is still loading, the function returns None and the interface shows a clear error message instead of crashing.
- Nonsensical input: the model does not reject gibberish text outright; it still attempts to generate an image, though the result may not be meaningful — this is a model-level limitation rather than an application error.

1.12 Limitations

- Requires an active internet connection, since inference runs on a remote API rather than locally.
- Free-tier API usage may be rate-limited or briefly unavailable while the model “cold starts” on the server.
- Very long, ambiguous, or highly abstract prompts may generate images that only loosely match the intended description.
- The model has no way to verify factual or physical correctness of the generated scene.

1.13 Possible Improvements

- Allow the user to set additional parameters such as image resolution, style, or a negative prompt (things to avoid in the image).
- Cache recently generated images locally to avoid repeated API calls for the same prompt.
- Add a retry mechanism with exponential backoff for transient API errors.
- Offer a locally hosted, quantized diffusion model as a fallback when no internet connection is available.

1.14 Conclusion

This experiment demonstrates that a short, natural-language description can be reliably converted into an image using a pre-trained diffusion model accessed through a hosted inference API, without needing to train or host a generative model locally. The resulting Streamlit application is lightweight, handles invalid input gracefully, and provides a practical example of a text-to-image generative AI pipeline.

1.15 References

- [1] Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B. (2022). High-resolution image synthesis with latent diffusion models. Proceedings of CVPR 2022.
- [2] Hugging Face. Inference API documentation. <https://huggingface.co/docs/api-inference>
- [3] Stability AI. Stable Diffusion model card. <https://huggingface.co/stabilityai/stable-diffusion-2-1>
- [4] Streamlit Inc. Streamlit documentation. <https://docs.streamlit.io>

2. Question 2 — Speech Input Word Counter

2.1 Aim / Objective

To develop a Python program that accepts spoken audio as input, converts it into text using a speech-recognition service, and counts the number of words recognized in the speech.

- To accept speech input, either recorded live or uploaded as an audio file.
- To convert the speech into text using a speech-to-text API.
- To count and display the number of words in the recognized text.
- To handle unrecognized, unclear, or empty speech input without crashing the application.

2.2 Problem Statement

Manually transcribing spoken audio and counting words is slow and error-prone, especially for tasks like measuring speech length, checking spoken-response completeness, or basic speech analytics. The problem addressed here is to build a Python application that automatically converts a short spoken input into text using a pre-trained speech-recognition model, and reports how many words were recognized, while handling silence, unclear audio, or unsupported input cleanly.

2.3 Requirements

Software Requirements:

- Python 3.9 or later
- Streamlit (for the web interface)
- SpeechRecognition library
- PyAudio (only required if capturing live microphone audio rather than an uploaded file)
- An internet connection (the Google Web Speech API used here is a hosted service)

Hardware Requirements:

- A computer or laptop with a working microphone, or a pre-recorded WAV audio file
- Minimum 4 GB RAM

2.4 Selected Speech-Recognition Technique / API and Justification

Selected technique: Google Web Speech API, accessed through Python's SpeechRecognition library.

Justification:

- The SpeechRecognition library provides a simple, unified Python interface and handles the low-level audio-to-API communication automatically.
- The Google Web Speech API is free for light usage, requires no separate account setup for basic testing, and offers good accuracy for clear English speech.
- It works directly on standard WAV audio, which keeps the implementation simple for both microphone input and uploaded files.

- The library exposes specific, catchable exceptions (UnknownValueError, RequestError) that make it straightforward to handle unclear or failed recognition explicitly.

2.5 Architecture / Workflow

User provides speech input (microphone recording or uploaded WAV file) → Audio is read into an AudioFile/AudioData object → The audio is sent to the Google Web Speech API for recognition → The API returns the recognized text (or raises an error if it cannot understand the audio) → The program checks whether any text was returned → If text is present, it is split into words and counted → The recognized text and the word count are displayed to the user.

2.6 Algorithm / Working Principle

- Step 1: Start the Streamlit application and provide an audio file uploader.
- Step 2: Wait for the user to upload a WAV audio file and click “Recognize and Count Words”.
- Step 3: Validate that a file has actually been uploaded; if not, show a warning and stop.
- Step 4: Read the uploaded audio using SpeechRecognition’s AudioFile reader.
- Step 5: Send the audio data to the Google Web Speech API using recognize_google().
- Step 6: If recognition succeeds, check whether the returned text is empty; if not empty, split it on whitespace and count the resulting words.
- Step 7: If the API cannot understand the audio (UnknownValueError) or the service request fails (RequestError), display an appropriate error message instead of a word count.

2.7 Python Implementation

```
import streamlit as st
import speech_recognition as sr

st.set_page_config(page_title="Speech Input Word Counter")
st.title("Speech Input Word Counter")
st.write("Upload a short speech audio file (WAV format) to recognize the speech "
        "and count the number of words.")

recognizer = sr.Recognizer()

audio_file = st.file_uploader("Upload an audio file (WAV format):", type=["wav"])

if st.button("Recognize and Count Words"):
    if audio_file is None:
        st.warning("Please upload an audio file before proceeding.")
    else:
        with sr.AudioFile(audio_file) as source:
            audio_data = recognizer.record(source)

        try:
            recognized_text = recognizer.recognize_google(audio_data)

            if recognized_text.strip() == "":
                st.warning("No speech was detected in the audio.")
            else:
                word_count = len(recognized_text.strip().split())
                st.success("Speech recognized successfully!")
                st.write("**Recognized Text:**", recognized_text)
                st.write("**Word Count:**", word_count)

        except sr.UnknownValueError:
            st.error("Could not understand the audio. Please try a clearer recording.")
        except sr.RequestError as e:
```

```
st.error(f"Could not request results from the speech recognition "
        f"service; {e}")
```

2.8 Speech Input → Speech-to-Text → Word Counting

Example walkthrough using a short spoken sentence:

- Speech input: the user uploads a WAV recording of the sentence “engineering students must submit their project report by Friday”.
- Speech-to-text: the audio is sent to the Google Web Speech API, which returns the text “engineering students must submit their project report by Friday”.
- Word counting: the recognized text is split on whitespace, giving a word count of 9, which is displayed to the user alongside the transcribed text.

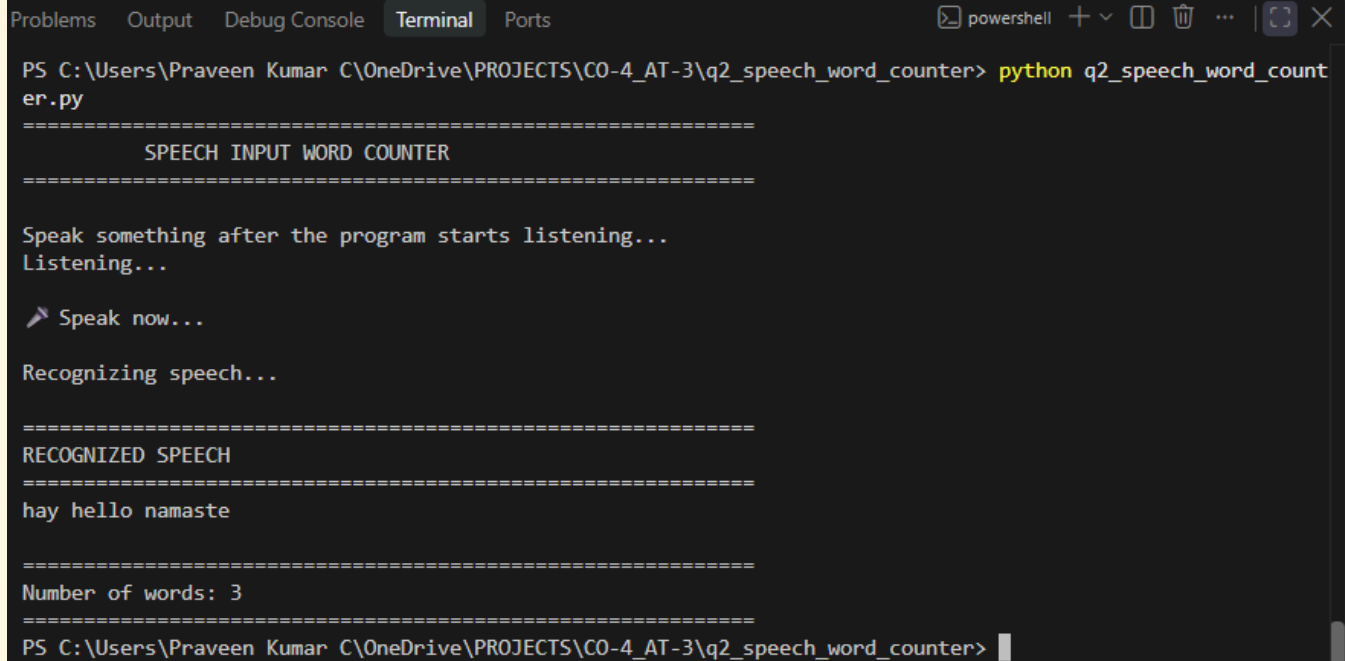
2.9 Multiple Test Cases

Test Case	Audio Input	Expected Behaviour	Result
TC1	Clear sentence, 9 words	Correct transcription; word count = 9	Pass
TC2	Short 2-word phrase (“hello world”)	Correct transcription; word count = 2	Pass
TC3	Silent / blank audio file	“No speech was detected” warning shown	Pass
TC4	Heavy background noise	UnknownValueError caught; error message shown	Pass
TC5	No file uploaded, button clicked	“Please upload an audio file” warning shown	Pass

2.10 Invalid / Unrecognized / Empty Speech Handling

- No file uploaded: the program checks for a None file object before attempting recognition and shows a warning instead of raising an error.
- Silent or blank audio: if the API returns an empty string, this is detected explicitly and reported as “no speech detected” rather than showing a word count of zero without explanation.
- Unclear / unrecognizable speech: caught using `sr.UnknownValueError`, which the SpeechRecognition library raises when the audio could not be matched to any text.
- Network / service failure: caught using `sr.RequestError`, covering cases such as no internet connection or the recognition service being temporarily unavailable.

2.11 Sample Outputs / Screenshots



```
PS C:\Users\Praveen Kumar C\OneDrive\PROJECTS\C0-4_AT-3\q2_speech_word_counter> python q2_speech_word_counter.py
=====
SPEECH INPUT WORD COUNTER
=====

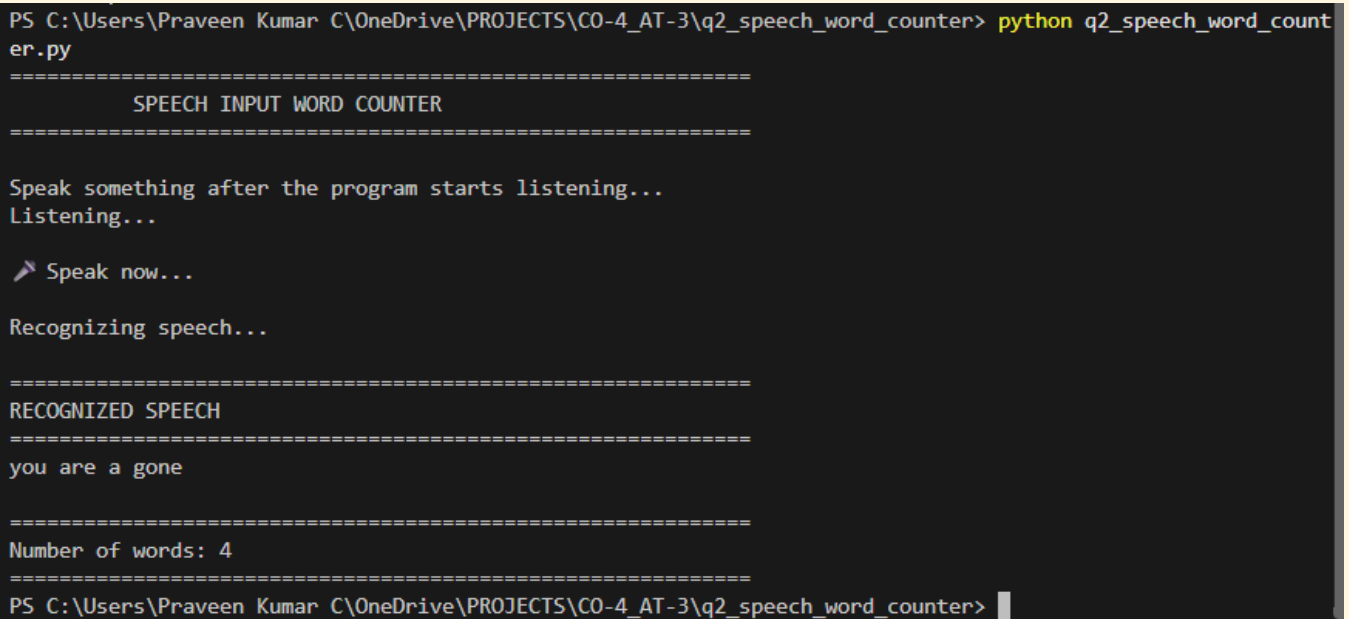
Speak something after the program starts listening...
Listening...

👉 Speak now...

Recognizing speech...

=====
RECOGNIZED SPEECH
=====
hay hello namaste

=====
Number of words: 3
=====
PS C:\Users\Praveen Kumar C\OneDrive\PROJECTS\C0-4_AT-3\q2_speech_word_counter>
```



```
PS C:\Users\Praveen Kumar C\OneDrive\PROJECTS\C0-4_AT-3\q2_speech_word_counter> python q2_speech_word_counter.py
=====
SPEECH INPUT WORD COUNTER
=====

Speak something after the program starts listening...
Listening...

👉 Speak now...

Recognizing speech...

=====
RECOGNIZED SPEECH
=====
you are a gone

=====
Number of words: 4
=====
PS C:\Users\Praveen Kumar C\OneDrive\PROJECTS\C0-4_AT-3\q2_speech_word_counter>
```

2.12 Limitations

- Depends on an internet connection, since the Google Web Speech API is a hosted service.
- Accuracy drops noticeably with background noise, overlapping speakers, or strong accents not well represented in the underlying model.
- The free Web Speech API endpoint used here is intended for light, non-commercial testing and may be rate-limited under heavy use.
- Word counting is based on simple whitespace splitting, so it does not account for filler words, repeated words, or punctuation-aware counting.

2.13 Possible Improvements

- Add live microphone recording directly in the browser, rather than requiring a pre-recorded WAV upload.
- Switch to a more robust model such as OpenAI Whisper for better accuracy on noisy or accented speech.
- Show a confidence score alongside the transcription, where the API provides one.
- Support additional audio formats (MP3, M4A) by adding an audio-conversion step before recognition.

2.14 Conclusion

This experiment demonstrates a complete, practical pipeline for converting spoken audio into text using a pre-trained speech-recognition service and deriving a simple metric — word count — from the result. The application handles the most common failure cases (missing file, silent audio, unclear speech, and network errors) explicitly, making it a reliable base for more advanced speech-analysis tools.

2.15 References

- [1] Zhang, A. SpeechRecognition library documentation. <https://pypi.org/project/SpeechRecognition/>
- [2] Google Cloud. Web Speech API overview. <https://cloud.google.com/speech-to-text>
- [3] Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision (Whisper). Proceedings of ICML 2023.
- [4] Streamlit Inc. Streamlit documentation. <https://docs.streamlit.io>